

Structured Data Extractor

Assessment - SDE

Language	Python or Node.js
LLM	Any (OpenAI, Anthropic, Gemini)

OVERVIEW

Users of your SaaS paste unstructured text into the product - a vendor email, a scanned receipt, a Slack message - and expect the system to pull out structured, database-ready fields automatically. Your task is to build the API that does this. The API accepts raw text, sends it to an LLM, and returns extracted fields with a confidence score for each one. Fields the model is uncertain about must be flagged for human review and must not be treated the same as fields the model is confident about. There is no frontend. Build the backend only.

Suggestions: Use Google AI studio to get gemini API and preferably use gemini-2.5-flash-lite

WHAT YOU NEED TO BUILD

1. Extraction endpoint

A single POST endpoint that accepts a raw text string and returns a structured JSON object containing the extracted fields, each with a value and a confidence score between 0.0 and 1.0. The extraction must use an LLM. You are responsible for designing the prompt such that the model returns per-field confidence scores reliably - not a single score for the whole response.

2. Review flag logic

Any field with a confidence score below 0.75 must be marked `needs_review: true` in the response. The overall extraction result must include a top-level `reviewed_required` boolean that is true if any field is flagged. This flag is what a downstream system would use to decide whether to auto-save the row or route it to a human.

3. Test inputs

Run your endpoint against all five sample inputs provided in the section below. Save each response to a file named `results.json`. The sample inputs are deliberately varied - some are clean, some are ambiguous, some are missing fields. Your system must handle all of them without crashing.

TARGET FIELDS

Extract the following fields from every input. Required fields must always appear in the response, even if the value is null and confidence is low.

Field	Type	Description	Required
vendor_name	string	Name of the vendor or sender	Yes
amount	number	Total monetary amount (numeric, no currency symbol)	Yes
currency	string	ISO 4217 currency code (e.g. INR, USD)	Yes
date	string	Date of the transaction in YYYY-MM-DD format	Yes
category	string	One of: food, travel, utilities, software, other	Yes
description	string	One-line summary of what the transaction is for	No
invoice_id	string	Invoice or reference number if present in the text	No

ENDPOINT SPECIFICATION

Method	Endpoint	Description
POST	/extract	Accepts { "text": string } and returns the structured extraction result.

The response must follow this exact shape:

```
{
  "review_required": true,
  "fields": {
    "vendor_name": { "value": "Swiggy", "confidence": 0.95, "needs_review": false },
    "amount":      { "value": 349, "confidence": 0.88, "needs_review": false },
    "currency":    { "value": "INR", "confidence": 0.91, "needs_review": false },
    "date":        { "value": null, "confidence": 0.20, "needs_review": true  },
  }
}
```

```
    "category":    { "value": "food",          "confidence": 0.82, "needs_review": false },
    "description": { "value": "Food order", "confidence": 0.78, "needs_review": false },
    "invoice_id":  { "value": null,          "confidence": 0.10, "needs_review": true  }
  }
}
```

SAMPLE INPUTS

Test your endpoint against all five inputs below. Include all five outputs in results.json.

#	Label	Input text	What to watch
1	Clean receipt	Paid ₹ 1,250 to AWS India Pvt Ltd on 12 March 2024. Invoice #INV-2024-0312. Services: cloud compute.	All fields present
2	Vendor email	Hi team, please find attached the invoice from Zoho Corp for ₹ 4,800 (software subscription, April 2024). Reference: ZHO-88821.	No explicit date in body
3	Slack message	hey can someone reimburse me for the uber last tuesday? it was like 340 rupees, went to the airport for the client meeting	Ambiguous date, no invoice
4	Scanned receipt	DELHI METRO RAIL CORP Token: 0042 Fare: INR 30 Date: 29-04-2024 From: Rajiv Chowk To: Hauz Khas	Unusual format, low amount
5	Ambiguous/garbage	meeting notes: discussed q2 targets, revenue up 12%, john to follow up with vendors next week	No transaction data at all

CONSTRAINTS

- Confidence scores must be per field, not a single score for the entire response. A prompt that asks the model to rate the overall extraction does not satisfy this requirement.
- The threshold for needs_review is 0.75. This must be applied consistently across all fields and all inputs.
- Input 5 contains no transaction data. Your system must return all required fields as null with low confidence. It must not crash or return a 500.
- Do not hardcode extraction logic. The LLM must do the extraction. Post-processing to normalise formats (e.g. dates, amounts) is permitted.

DELIVERABLES

- Working POST /extract endpoint, runnable from your README instructions.
- README: setup steps, your LLM and model choice with reasoning, and an explanation of how your prompt elicits per-field confidence scores.
- results.json: the response from your endpoint for all five sample inputs above, in an array.

EVALUATION

Criterion	Weight	What we look for
Extraction accuracy - required fields correctly pulled from clean inputs	30%	Inputs 1 and 2 should have high confidence across most fields.
Per-field confidence design - prompt produces genuine per-field scores	25%	Scores are not identical across all fields; ambiguous inputs score lower.
Review flag logic - threshold applied correctly in all five responses	20%	Input 5 must have review_required: true. Input 1 likely should not.
Failure handling - no crashes on ambiguous or empty inputs	15%	Input 5 returns a valid response, not an error.
README - prompt strategy clearly explained	10%	How does the prompt get the model to return per-field confidence?

Instructions

Kindly create a private GitHub repository named: {yourName_Nestack_Submission} (e.g., roy_Nestack_Submission) and add the following contributors for evaluation:

- bishal@nestack.com
- sannidhya@nestack.com
- sanjay@nestack.com

Your submission must include:

- A ZIP file of your complete codebase
- A live deployment link
- Any necessary credentials if your project includes authentication

Additionally, ensure your repository includes a README.md with:

- The deployment link clearly mentioned
- Setup and run instructions if applicable

NOTE: PUBLIC REPOSITORIES ARE DISQUALIFIED FROM THE HIRING PROCESS